CSCI 2406: Topic 04

Integer Arithmetic and Performance Basics

Fixed-width arithmetic, condition flags, shifts, and CPU execution-time analysis

Computer Organization & Architecture | AArch64 Reference

Topic 4 Outline

Topic Objective

Understand how fixed-width integer arithmetic works at the machine level, how condition flags support later control flow, and how instruction count, CPI, and clock rate determine execution time.

- **Section 1**: Fixed-width addition, subtraction, and interpretation
- **Section 2**: Condition flags and AArch64 comparison behavior
- **Section 3**: Multiplication, division, and shifts
- **Section 4**: Clock rate, CPI, and execution time
- **Section 5**: Summary and self-test

Section 01

Fixed-Width Integer Arithmetic

Why Integer Arithmetic Matters

Integer Operations Are Everywhere

Integer arithmetic is not limited to explicit calculator-like expressions. It appears constantly in address calculation, loop control, array indexing, comparisons, and resource accounting.

Program-Level Examples

- Increment loop counters.
- Compute array offsets.
- Compare bounds and indices.
- Update pointer values.

Hardware-Level Consequence

- Arithmetic uses fixed-width registers.
- Only the destination-width result bits are retained.
- Flags record selected facts about results.
- Interpretation is supplied by software.

Arithmetic Operates on Bit Patterns

Core Principle

The ALU operates on finite-width bit patterns. Signedness is not a property of the adder itself; it is a property of how software interprets the bits and which branch conditions it later uses.

Same Instruction

```
add x0, x1, x2
```

- Adds two 64-bit values.
- Produces one 64-bit result.
- Does not by itself say signed or unsigned.

Different Readings

- Unsigned: magnitude from 0 to $2^{64} - 1$.
- Signed: two's-complement range from -2^{63} to $2^{63} - 1$.
- Flags help evaluate special cases.

Binary Addition and Carry

Column Addition

Binary addition proceeds bit by bit, carrying into the next higher column when needed.

```
  1111
+ 0001
——
1 0000
```

Unsigned Meaning

- In 4 bits, only the lower `0000` remains.
- The extra leading bit is carry-out.
- For unsigned arithmetic, carry-out means the mathematical result exceeded the available range.

Fixed-Width Result

An n-bit register keeps only n result bits. Extra information may be reflected in condition flags.

Unsigned Ranges

Range Formula

An n-bit unsigned integer represents values from 0 to $2^n - 1$.

Width	Smallest	Largest
8 bits	0	255
32 bits	0	$2^{32} - 1$
64 bits	0	$2^{64} - 1$

Fixed-Width Result

If unsigned addition produces a carry out of the most-significant bit, the destination stores only the low n bits of the result.

Two's-Complement Subtraction

Hardware Idea

Subtraction can reuse the same adder by adding the two's-complement negation of the second operand.

$$A - B = A + (\sim B) + 1$$

AArch64 Example

```
sub x0, x1, x2
```

- Computes `x1 - x2`.
- Writes the fixed-width result to `x0`.
- Does not update NZCV flags.

Important Interpretation

- Negative values use two's-complement representation.
- The same adder can perform addition and subtraction.
- NZCV records facts needed for later signed and unsigned reasoning.

Signed vs. Unsigned Interpretation

Same Bits, Different Values

The same bit pattern can have different numeric meanings depending on whether software treats it as unsigned or signed two's complement.

8-bit pattern	Unsigned	Signed two's complement
`01111111`	127	127
`10000000`	128	-128
`11111111`	255	-1

Connection to AArch64

The instruction result is a bit pattern. The choice of signed or unsigned branch determines how flags are interpreted later.

Section 02

Condition Flags and Comparisons

Why Condition Flags?

Arithmetic Produces More Than a Result

A fixed-width arithmetic operation produces a register result, but later instructions may also need to know whether that result was zero, negative, produced a carry-out, or caused signed overflow.

- These properties are recorded in four condition flags: `N`, `Z`, `C`, and `V`.
- Flag-setting arithmetic updates NZCV while also computing the normal register result.
- Comparison instructions update NZCV without keeping the arithmetic result.
- Conditional branches then test these flags to make control-flow decisions.

Why This Matters

NZCV connects arithmetic to decisions such as equality, signed ordering, and unsigned ordering.

NZCV Condition Flags

Processor Status

AArch64 maintains architectural processor state in **PSTATE**. Four condition-code bits within that state record selected properties of arithmetic and logical results and are collectively called **NZCV**.

- **N**: most-significant bit of the result is 1.
- **Z**: result is exactly zero.
- **C**: records carry-out for addition and supports unsigned comparison after subtraction.
- **V**: indicates signed two's-complement overflow.

NZCV Representation

When NZCV is accessed as a system-register value, the four flags occupy bits 31–28 in the order `N Z C V`.

Flag-Setting Arithmetic

No Flag Update

```
add x0, x1, x2
sub x0, x1, x2
```

- Write a register result.
- Leave NZCV unchanged.

Flag Update

```
adds x0, x1, x2
subs x0, x1, x2
```

- Write a register result.
- Update NZCV from that operation.

AArch64 Detail

Flag updates are provided by specific instruction forms such as `adds` and `subs`; this is not a general rule of simply adding an `s` to every mnemonic.

N: Negative Flag

When N Is Set

For a flag-setting arithmetic operation, `N=1` when the most-significant bit of the destination-width result is 1.

Example

```
3 - 5 = 1110
```

- In 4-bit two's complement, `1110` represents -2.
- The sign bit is 1, so `N=1`.

AArch64 Context

```
subs x0, x1, x2
```

- Writes the subtraction result to `x0`.
- Updates NZCV from that result.
- Ordinary sub leaves NZCV unchanged.

Important Limitation

Signed less-than uses `N` together with `V`; `N` alone is not sufficient.

Z: Zero Flag

When Z Is Set

`Z=1` when every bit of the destination-width result is zero.

Arithmetic Example

```
0101 - 0101 = 0000
```

- The result is exactly zero.
- Therefore `Z=1`.

Comparison Example

```
cmp x0, x1
```

- `cmp` evaluates `x0 - x1` for flags.
- If `x0 == x1`, the difference is zero.
- Therefore `Z=1`.

Branch Connection

After a comparison, `b.eq` tests `Z=1`; `b.ne` tests `Z=0`.

C: Carry Flag

Addition

For addition, `C=1` when the fixed-width addition produces a carry out of the most-significant bit. Only the low n result bits are stored in an n-bit destination.

4-Bit Addition

```
    1111 + 0001
    = 1 0000
```

- Stored result: `0000`.
- Carry out: 1.
- Therefore `C=1`.

Subtraction Uses the Same Adder

$$A - B = A + (\sim B) + 1$$

```
5 - 3:  0101 + 1101 = 1 0010
3 - 5:  0011 + 1011 = 1110
```

- Carry out $\Rightarrow$ `C=1` and $5 \geq 3$.
- No carry out $\Rightarrow$ `C=0` and $3 < 5$.

C After Subtraction

Unsigned Interpretation

For the subtraction $A - B$, the carry flag provides information about the unsigned relationship between the two operands.

Condition	Carry flag	Unsigned interpretation
$A \geq B$	`C=1`	The fixed-width subtraction produces a carry out.
$A < B$	`C=0`	The fixed-width subtraction produces no carry out.

AArch64 Comparison Use

After `cmp x0, x1`, `C=1` corresponds to `x0 >= x1` for unsigned values, while `C=0` corresponds to `x0 < x1`.

V: Signed Overflow Flag

Signed Addition Rule

For signed addition, overflow occurs when two operands with the same sign produce a result with the opposite sign.

Positive + Positive

```
0111 + 0001 = 1000
```

- $7 + 1 = 8$ cannot be represented in 4-bit signed arithmetic.
- Two positive operands produced a negative result pattern.
- Therefore `V=1`.

Negative + Negative

```
1000 + 1111 = 0111
```

- $-8 + (-1) = -9$ cannot be represented in 4 bits.
- Two negative operands produced a positive result pattern.
- Therefore `V=1`.

Adding operands with opposite signs does not produce signed overflow in addition.

Signed Overflow in Subtraction

Subtraction Rule

For signed subtraction `A - B`, overflow can occur only when `A` and `B` have opposite signs and the result sign differs from the sign of `A`.

Positive Minus Negative

A positive value minus a negative value should become more positive. If the fixed-width result becomes negative, signed overflow occurred.

Negative Minus Positive

A negative value minus a positive value should become more negative. If the fixed-width result becomes positive, signed overflow occurred.

Different Roles

`C` supports unsigned reasoning after subtraction; `V` records whether the signed subtraction exceeded the representable signed range.

Comparison with cmp

cmp Is Flag-Setting Subtraction

```
cmp x0, x1
subs xzr, x0, x1
```

- The processor evaluates the same arithmetic needed for `x0 - x1`.
- The numerical subtraction result is discarded by writing it to `xzr`.
- NZCV records properties of the subtraction.
- A later conditional branch interprets those flags as equality, signed ordering, or unsigned ordering.

Important Point

`cmp` does not itself mean “signed compare” or “unsigned compare.” The later condition code determines how NZCV is interpreted.

Comparisons and Branch Interpretation

Branch	Interpretation	Condition after `cmp x0, x1`
`b.eq`	equal	`Z=1`
`b.ne`	not equal	`Z=0`
`b.lt`	signed $<$	`N != V`
`b.ge`	signed $\geq$	`N == V`
`b.lo`	unsigned $<$	`C=0`
`b.hs`	unsigned $\geq$	`C=1`

Logical Flow

`Z` answers equality, `C` supports unsigned ordering, and the combination of `N` and `V` supports signed ordering.

Section 03

Multiplication, Division, and Shifts

Multiplication and Division Basics

Architectural Category

Multiply and divide read registers and write register results; latency is implementation-dependent.

Multiply

```
mul x0, x1, x2
```

- Computes a register product.
- May use a distinct multiply unit internally.
- Latency is implementation-dependent.

Divide

```
udiv x0, x1, x2
sdiv x0, x1, x2
```

- `udiv`: unsigned division.
- `sdiv`: signed division.
- Often more expensive than add/sub.

Shifts and Powers of Two

Shift	Meaning	Typical interpretation
`lsl #n`	shift left, zeros enter right	multiply by 2^n if overflow is ignored
`lsr #n`	logical right, zeros enter left	unsigned division by powers of two
`asr #n`	arithmetic right, sign bit replicated	preserves signed negative values
`ror #n`	rotate right, bits move around ends	bit manipulation, not division

Key Distinction

`lsr` and `asr` differ when the top bit is set. That difference matters for unsigned versus signed interpretation.

Shift Example

Logical Shift Right

```
lsr x0, x1, #1
```

- Zero enters the top bit.
- Good for unsigned bit patterns.
- Does not preserve a negative sign.

Arithmetic Shift Right

```
asr x0, x1, #1
```

- Original sign bit is copied.
- Preserves signed interpretation.
- Useful for signed right shifts.

Qualification

Shifts can resemble multiplication or division by powers of two, but finite width, overflow, and signed rounding behavior must be considered.

Section 04

Processor Performance Basics

Execution Time and Performance

Primary Metric

For a fixed workload, performance is the inverse of execution time.

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

Latency

- Time to complete one task.
- Important for response time.
- Example: seconds per program run.

Throughput

- Work completed per unit time.
- Important for pipelines and servers.
- Example: completed instructions per cycle.

The Iron Law of Processor Performance

CPU Execution-Time Equation

$$T_{\text{CPU}} = \text{IC} \times \text{CPI} \times T_{\text{clk}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$$

Term	Meaning	Influenced by
IC	dynamic instruction count	algorithm, compiler, ISA
CPI	cycles per instruction	pipeline, memory, branches
T_{clk}	seconds per cycle	circuit timing, clock frequency

Instruction Count Is Dynamic

Dynamic Instruction Count

Dynamic instruction count is the number of instructions actually executed, not merely the number of instructions visible in a source or assembly listing.

- Loops multiply the effect of a small instruction sequence.
- Branches choose which instructions execute.
- Function calls, recursion, and input data can change the count.
- Optimizing one program region may not matter if it rarely executes.

Simple Example

A three-instruction loop body executed N times contributes approximately $3N$ dynamic instructions, before considering branch and loop-control instructions.

CPI and IPC

Cycles Per Instruction

$$\text{CPI} = \frac{\text{cycles}}{\text{instructions}}$$

- Average over a workload.
- Can include stalls and cache misses.
- Lower is usually better.

Instructions Per Cycle

$$\text{IPC} = \frac{\text{instructions}}{\text{cycles}}$$

- Average completed instructions per cycle.
- Can exceed 1 on superscalar cores.
- Higher is usually better.

No Universal Cycle Count

Instruction latency and throughput are microarchitecture-dependent, not fixed by the AArch64 ISA alone.

Average CPI from an Instruction Mix

Weighted Average

When instruction classes have different average costs, overall CPI depends on the dynamic instruction mix.

$$\text{Average CPI} = \sum_i (\text{fraction}_i \times \text{CPI}_i)$$

Class	Fraction	CPI	Contribution
A	50%	1	0.50
B	30%	2	0.60
C	20%	4	0.80

Result

$$\text{Average CPI} = 1.90$$

Clock Frequency Is Not Performance

Common Mistake

A higher clock rate does not automatically mean a shorter runtime. Instruction count and CPI matter just as much.

Processor	Clock	CPI	Time / instruction
A	4.0 GHz	2.0	0.50 ns
B	3.0 GHz	1.0	0.333 ns

Interpretation

Processor B has a lower frequency, but each instruction takes fewer cycles on average, so its average time per instruction is lower for this workload.

Worked Performance Example

Machine A

- IC $= 1.0 \times 10^9$
- CPI $= 2.0$
- $f = 4.0$ GHz

$$T_A = \frac{1.0 \times 10^9 \times 2.0}{4.0 \times 10^9} = 0.50 \text{ s}$$

Machine B

- IC $= 1.2 \times 10^9$
- CPI $= 0.8$
- $f = 2.5$ GHz

$$T_B = \frac{1.2 \times 10^9 \times 0.8}{2.5 \times 10^9} = 0.384 \text{ s}$$

Speedup

$$\text{Speedup} = \frac{0.50}{0.384} \approx 1.30\times$$

Section 05

Summary and Self-Test

Topic 4 Summary

- Integer arithmetic operates on fixed-width bit patterns.
- Signedness comes from interpretation, not from the adder itself.
- AArch64 records selected arithmetic properties in the NZCV condition flags within PSTATE.
- `N` records the most-significant result bit; `Z` records whether the result is zero.
- `C` records carry-out for addition and supports unsigned ordering after subtraction.
- `V` records signed two's-complement overflow.
- `cmp` updates NZCV without retaining the subtraction result.
- Signed and unsigned conditional branches interpret the same NZCV flags differently.
- Execution time depends jointly on instruction count, CPI, and clock period.

Self-Test 1/2

1. Why do `add` and `sub` not by themselves determine signedness?
2. How can subtraction be implemented using addition and two's complement?
3. What does the `N` flag record?
4. Under what condition is `Z=1`?
5. What does carry-out mean for fixed-width unsigned addition?
6. After `cmp x0, x1`, what does `C=1` mean for unsigned comparison?

Self-Test 2/2

7. For signed addition, when is `V=1`?
8. What is the difference between `add` and `adds`?
9. Why can `cmp x0, x1` be understood as `subs xzr, x0, x1`?
10. Which flags are used for signed less-than?
11. Which flag is used for unsigned less-than after `cmp`?
12. Why can a lower-frequency processor outperform a higher-frequency processor?

Wrap-Up and Next Steps

Next Topic: AArch64 Arithmetic, Logic, Shifts, and Flags

The next lecture moves from arithmetic foundations to concrete AArch64 instruction forms and bit-level manipulation patterns.

- Arithmetic and logical instruction families
- Aliases such as `cmp` and `tst`
- Logical masks and shifts
- Immediate operands and assembler conveniences